

TP SOC-IP, 4 séances (3IS, 2025-26)

Réalisation d'un système MicroBlaze pour animer des "messages lumineux"

Logiciels :

- Vivado 19.1 + IP-Integrator, SDK 19.1

Plateforme :

- Basys3 de Digilent, **FPGA** : Artix®-7 de Xilinx, part number **XC7A35TCPG236C-1**

Objectif du TP :

Il s'agit de réaliser une animation de messages lumineux en utilisant les segments des afficheurs 7-segments présents sur la carte *Basys3 de Digilent*. Cela consiste à produire un mouvement lumineux sur les quatre afficheurs nommés **SEG0 à SEG3**.

Le sens du déplacement, de gauche à droite ou de droite à gauche fera partie des options à programmer. Nous allons parvenir à la réalisation complète de l'animation en effectuant les différentes étapes ci-après.

- ❖ **NB** : Il est recommandé de développer, à chaque fois que c'est possible, une fonction de test pour chaque étape. Prenez le temps nécessaire pour insérer des commentaires explicatifs devant le code de chaque étape réalisée.

Remarque importante :

- ❖ Appelez l'enseignant pour valider chaque étape.
- ❖ Un compte rendu (+ les sources) doit être fourni à la fin de la dernière séance.
- ❖ Choisissez vous-même les prototypes des fonctions (à commenter en quelques lignes)

Partie I : ANIMATION D'UN MESSAGE LUMINEUX (1 séance)

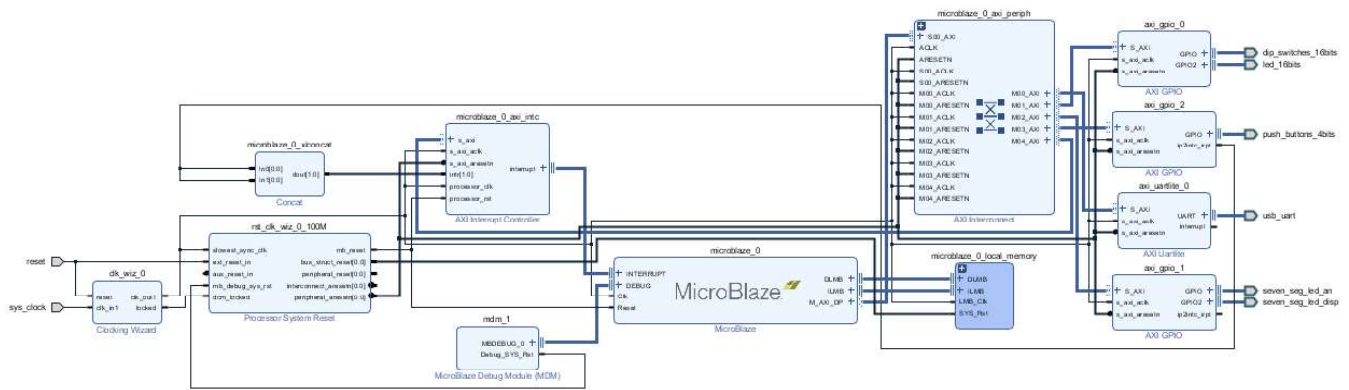
1. Réalisation d'un système numérique MicroBlaze

Commencez par créer un projet Vivado, le nommer « *TP_ublaze_3IS* », en choisissant Basys3 comme kit de développement (Basys3 board : xc7a35tcpg236-1). Pensez à sélectionner VHDL comme langage de description par défaut. Dans un premier temps, nous allons créer un système numérique autour du microcontrôleur MicroBlaze qui est une IP softcore mise à disposition par Xilinx.

Voici les étapes à réaliser dans *IP Integrator* pour créer un design nommé « *ublaze_system* » :

- A partir de l'onglet « *Board* », ajouter un bloc d'horloge « *System Clock* ».
- Ajouter une instance de l'IP MicroBlaze : augmenter la mémoire à 128 KB, puis activer l'option « *Interrupt Controller* » pour intégrer le contrôleur d'interruption « *INTC* ».
- A partir de l'onglet « *Board* », ajouter les IP suivantes : *LEDs*, *Switches*, *Push Buttons* (activer l'interruption), *Reset*, *USB UART* et les afficheurs (*Anodes + Segments*).
- Ajouter une IP de type Timer (*AXI Timer*).
- Lancer la réalisation automatique des connexions puis compléter l'opération en câblant manuellement les interruptions. Vérifier la validité du schéma : « *Validate Design* ».
- Générer le Toplevel : « *Create HDL Wrapper...* », puis analyser les fichiers VHDL obtenus (observer l'entity et vérifier que tous les ports d'entrées/sorties sont présents).
- Lancer la génération du « *Bitstream* », cette étape va durer un peu plus que cinq minutes. Ne pas paniquer à cause du temps long que prend la synthèse FPGA ; si la description hardware de votre système est bien faite vous n'aurez pas à refaire cette étape avant longtemps.

Le design dans « *Diagram* » devrait ressembler à ceci :



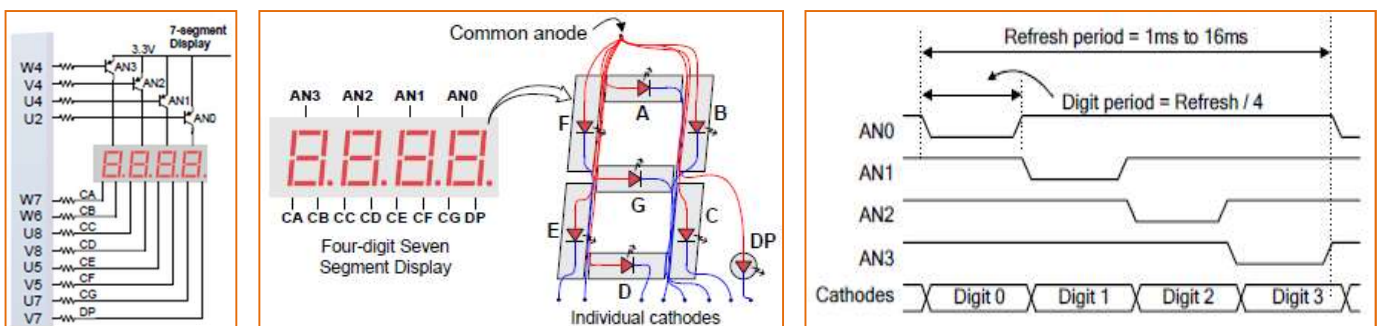
Dans SDK, créer un projet nommé « **Test_IP** ». Ecrire, en utilisant les pointeurs sur les IP, les fonctions ci-dessous (choisir les prototypes). Tester les fonctions dans la boucle *while* du **main()**.

- **Test_leds()** : pour allumer une LED sur deux
- **sw2leds()** : pour afficher l'état des LEDs selon l'état des switches
- **Cheniller()** : pour réaliser un chenillard avec les 16 LEDs (une seule LED allumée qui rebondit sur les extrémités). Utiliser **usleep()** pour rythmer le mouvement des LEDs (penser à inclure la directive **#include "sleep.h"**)

❖ **NB** : Appelez le professeur présent pour vérifier le bon fonctionnement. Il peut vous expliquer comment utiliser les bibliothèques de haut niveau HAL pour faire autrement. Consulter les prototypes des fonctions : **XGpio_Initialize()**, **XGpio_SetDataDirection()**, **XGpio_DiscreteRead()** et **XGpio_DiscreteWrite()**

2. Contrôle des 7 Segments par le *MicroBlaze*

Ne disposant pas encore de l'IP matérielle adaptée pour contrôler les 7 segments, nous allons le prendre en charge par le processeur **MicroBlaze**. Comme le montrent les figures ci-dessous, le bus de données est commun à tous les 7 segments. La sélection de l'afficheur à utiliser à un instant donné se fait en forçant son anode à '0'. Le respect des chronogrammes de la figure ci-dessous permet d'obtenir un affichage différencié sur les quatre afficheurs.



- Dans SDK, créer un nouveau projet software nommé « **Test_SEG7** ». Ecrire une fonction, appelée **sw2seg()**, pour afficher en hexadécimal la valeur composée par les switches (nous disposons de 16 switches au total et de 4 afficheurs).

❖ **NB** : Pour ce faire, déclarer une table de codage des segments et utiliser la fonction **usleep()** afin de respecter les timing (après avoir ajouté la directive suivante : **#include "sleep.h"**)

3. Animation du message « HELLO 2025 »

Maintenant que vous arrivez à contrôler les 7 segments avec *Microblaze*, nous allons passer à l'animation du message « HELLO 2025 ». L'objectif final à atteindre est d'offrir à l'utilisateur la possibilité de faire défiler un message de gauche à droite, de droite à gauche et avec une vitesse réglable.

- Dans SDK, créer un nouveau projet nommé « *animer_hello* ». Ecrire une fonction, appelée *hello2seg()*, pour valider le codage en segments du message. Pour ce faire, utiliser un tableau de la taille du message et y écrire le code de chaque caractère de « HELLO 2025 ». Le test de validation de chaque code peut se faire sur les 4 afficheurs en même temps (les 4 anodes activées).

NB : Pensez à utiliser *usleep()* pour maintenir l'affichage de chaque caractère pendant un temps suffisant.

La figure ci-contre, montre les 10 premières combinaisons à afficher dans le cas d'un défilement de *droite*→*gauche*.

- Toujours dans SDK, écrire une fonction, *afficher_hello(int position)*, pour afficher une partie du message sur les 4 afficheurs du kit Basys3. La fonction prend comme argument la position du premier caractère. Exemple : si *position=2*, "LLO " est affiché (n°5, voir ci-contre) →→→→
- Ecrire une nouvelle fonction, *Animer_message(int sens)*, pour animer la totalité du message. En se servant d'une variable globale pour fixer le sens, valider le défilement de *droite* à *gauche* (←) et de *gauche* à *droite* (→).
- Compléter la fonction précédente en mettant en place un défilement continu du message.

0				H
1			H	E
2		H	E	L
3	H	E	L	L
4	E	L	L	O
5	L	L	O	
6	L	O		2
7	O		2	0
8		2	0	2
9	2	0	2	5

Questions

1. Que pensez-vous de la difficulté à fixer les délais pour faire varier la vitesse de défilement tout en assurant un affichage stable en respectant les contraintes de la persistance rétinienne ?
2. Quelle solution préconisez-vous pour améliorer l'affichage du système ?

Partie II : AMELIORATION DU SYSTEME (1 séance)

Deux améliorations constituent la suite du TP, elles ont pour objectif de décharger le processeur **MicroBlaze** de la gestion des temporisations :

- 1) Utilisation d'un *Timer* pour animer le défilement périodique (utilisation des interruptions).
- 2) Création d'une interface matérielle spécifique pour contrôler les afficheurs

4. Utilisation d'un **Timer** pour rythmer le défilement

On se propose dans cette étape d'intégrer au système **MicroBlaze** un *Timer* pour contrôler la vitesse de défilement du message. Le *Timer* doit être initialisé pour obtenir une temporisation fixe de **0.125** secondes (et génère donc 8 interruptions par seconde).

- Ajoutez au design « *ublaze_system* » une IP de type Timer (*AXI Timer*)
- Ecrivez et tester deux fonctions pour cette étape :
 - *Timer_init()* pour initialiser le *Timer*
 - *Timer_isr()* comme fonction d'interruption du *Timer* pour faire clignoter les LEDs.

Indications :

Commencez par lire la documentation en ligne des pilotes disponibles dans le projet BSP de la suite SDK. Vous trouverez dans le fichier `microblaze_0/libsrc/tmrctr_vXX/src/xtrmctr.c` un exemple de fonctions composant un driver HAL (projet BSP). Vous y trouverez les fonctions suivantes :

- *XTmrCtr_Initialize()* pour initialiser l'objet XTmrCtr. La valeur ID du périphérique est nécessaire à cette fonction, vous la trouverez dans le fichier « *xparameters.h* » dans BSP. Vous devez vérifier la valeur de retour de cette fonction !
- *XTmrCtr_SetOptions()* pour définir les options du Timer. Vous devez régler la valeur de la temporisation, activer l'interruption (utilisez des indicateurs *XTC_DOWN_COUNT_OPTION* et *XTC_INT_MODE_OPTION* de `microblaze_0/libsrc/tmrctr_vXX/src/xtrmctr.c`).
- *XTmrCtr_SetResetValue()* pour définir la valeur initiale du compteur.
- *XTmrCtr_Start()* pour démarrer le Timer.
- *XTmrCtr_IsExpired()* pour vérifier si le Timer a atteint zéro (mauvaise alternative à l'interruption)
- *XTmrCtr_Reset()* pour recharger et réinitialiser le Timer.

5. Contrôle de la vitesse et du sens de défilement

- Utiliser 4 Switches, **SW[3..0]**, pour contrôler manuellement la vitesse de défilement. La valeur des Switches fixe la période en multiple de la temporisation du Timer.
- Utiliser **SW15** pour sélectionner le sens du défilement de **droite → gauche** et **gauche → droite**.
- Si le temps le permet, utiliser **un des boutons poussoirs** pour contrôler le sens du défilement.

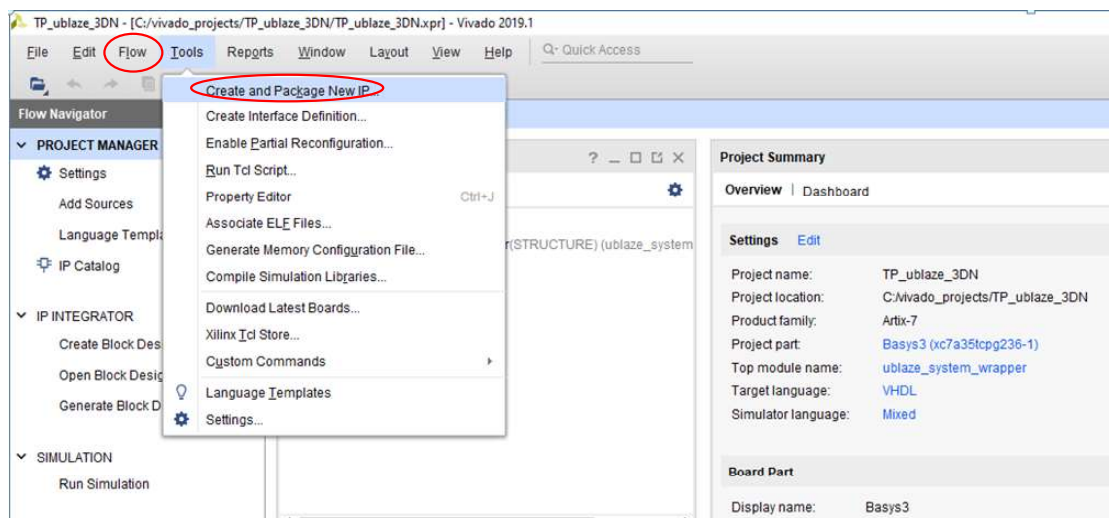
Partie III : SEG7_IP (2 séances)

Conception d'une interface matérielle adaptée (IP Hard)

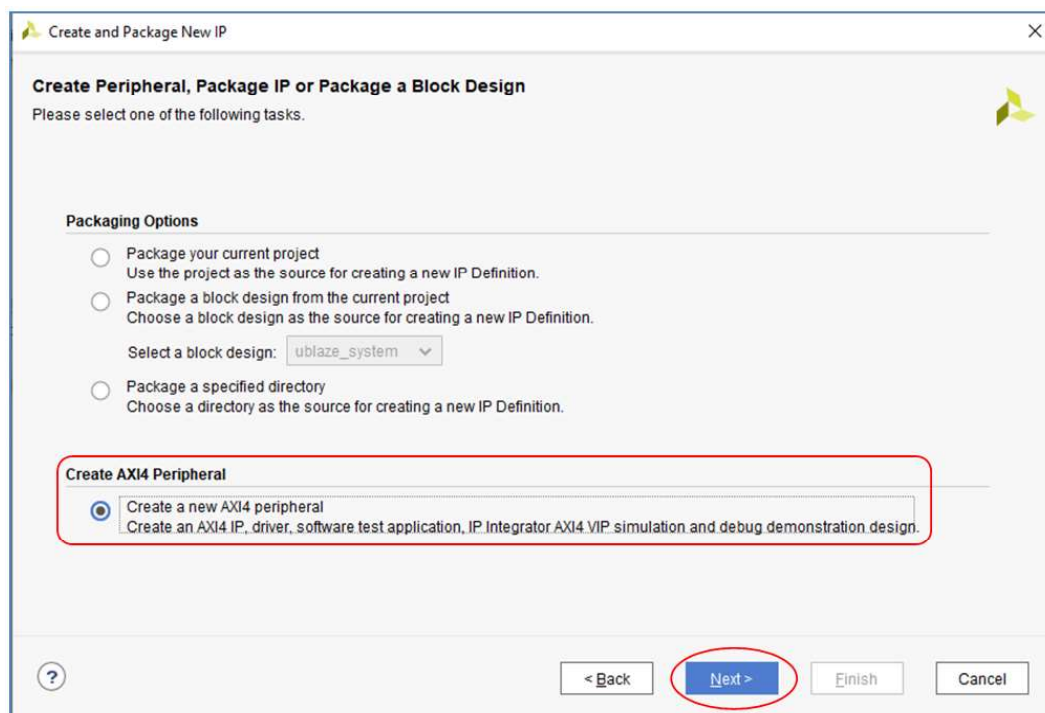
Cette partie va vous permettre de concevoir et de mettre au point une interface matérielle sur mesure pour contrôler efficacement les afficheurs 7 segments de la carte **Basys3**. L'utilitaire à utiliser dans Vivado vous guide tout au long du processus d'ajout d'un bloc matériel personnalisé, écrit en langage HDL (VHDL, Verilog, SystemC, HLS, ...), au microprocesseur à l'aide du bus AXI. Vous ajouterez le matériel contrôlant l'affichage des 7 segments en multiplexant les signaux de données des afficheurs. Le squelette du code HDL du contrôleur d'affichage 7 segments sera généré automatiquement par Vivado, il vous suffira de le modifier pour ajouter votre code HDL dans l'interface esclave AXI.

6. Ajout d'un nouveau bloc matériel (hardware IP)

- Sélectionnez le menu *Tools/Create and Package New IP* et suivez le guide



- L'assistant de création d'un nouveau bloc démarre, sélectionnez Créer un périphérique AXI :



- Dans la fenêtre suivante, vous pouvez ajouter le nom du nouveau bloc IP et un bref descriptif de ce que réalise le bloc comme fonction. Cliquer sur **Suivant** pour continuer :

Create and Package New IP

Peripheral Details
Specify name, version and description for the new peripheral

Name: SEG7_IP

Version: 1.0

Display name: SEG7_IP_v1.0

Description: My new AXI IP (SEG7_IP : pour contrôler 4 sept segments)

IP location: C:\vivado_projects\ip_repo

☐ Overwrite existing

Chemin du dépôt des IP

< Back Next > Finish Cancel

- Dans la fenêtre des **interfaces** qui s'ouvre, fixer le nombre de registres à 4. Cela permet de réserver un registre de donnée pour stocker la valeur à afficher sur chaque digit dans un registre séparé. Ces 4 emplacements mémoire de 32 bits seront disponibles pour le processeur en lecture et en écriture (nous verrons plus tard que nous n'utiliserons que 8 bits sur les 32). Cliquer sur **Suivant** pour continuer ...

Create and Package New IP

Add Interfaces
Add AXI4 interfaces supported by your peripheral

☐ Enable Interrupt Support

Interfaces

- S00_AXI

Name: S00_AXI

Interface Type: Lite

Interface Mode: Slave

Data Width (Bits): 32

Memory Size (Bytes): 64

Number of Registers: 4 [4..512]

< Back Next > Finish Cancel

- Dans la dernière fenêtre, qui résume le processus d'ajout d'un nouveau bloc IP, laissez l'option par défaut pour « **ajouter l'IP au dépôt** » et cliquez sur **Terminer**. Votre nouveau bloc IP sera ajouté immédiatement au catalogue IP.

Create and Package New IP

Create Peripheral

Peripheral Generation Summary

1. IP (xilinx.com:user:SEG7_IP:1.0) with 1 interface(s)
2. Driver(v1_00_a) and testapp more info
3. AXI4 VIP Simulation demonstration design more info
4. AXI4 Debug Hardware Simulation demonstration design more info

Peripheral created will be available in the catalog :
C:\vivado_projects\ip_repo

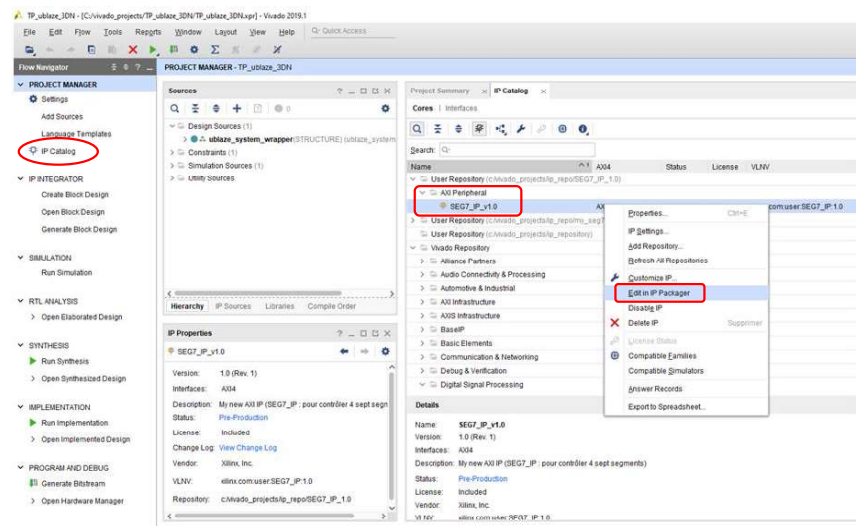
Next Steps:

- ☒ Add IP to the repository
- ☐ Edit IP
- ☐ Verify Peripheral IP using AXI4 VIP
- ☐ Verify peripheral IP using JTAG interface

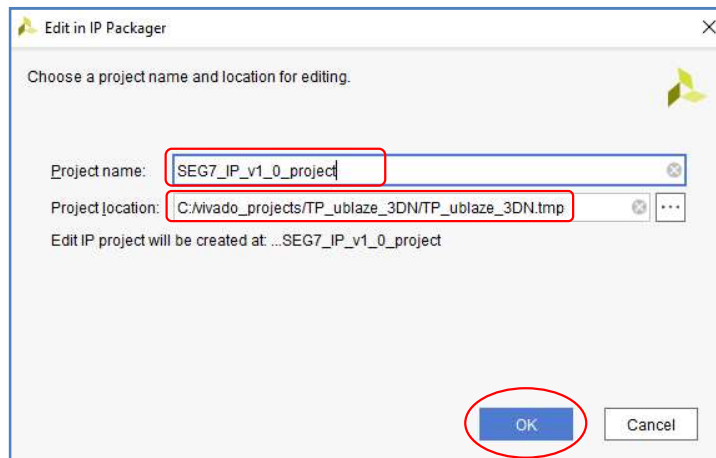
Click Finish to continue

< Back Next > Finish Cancel

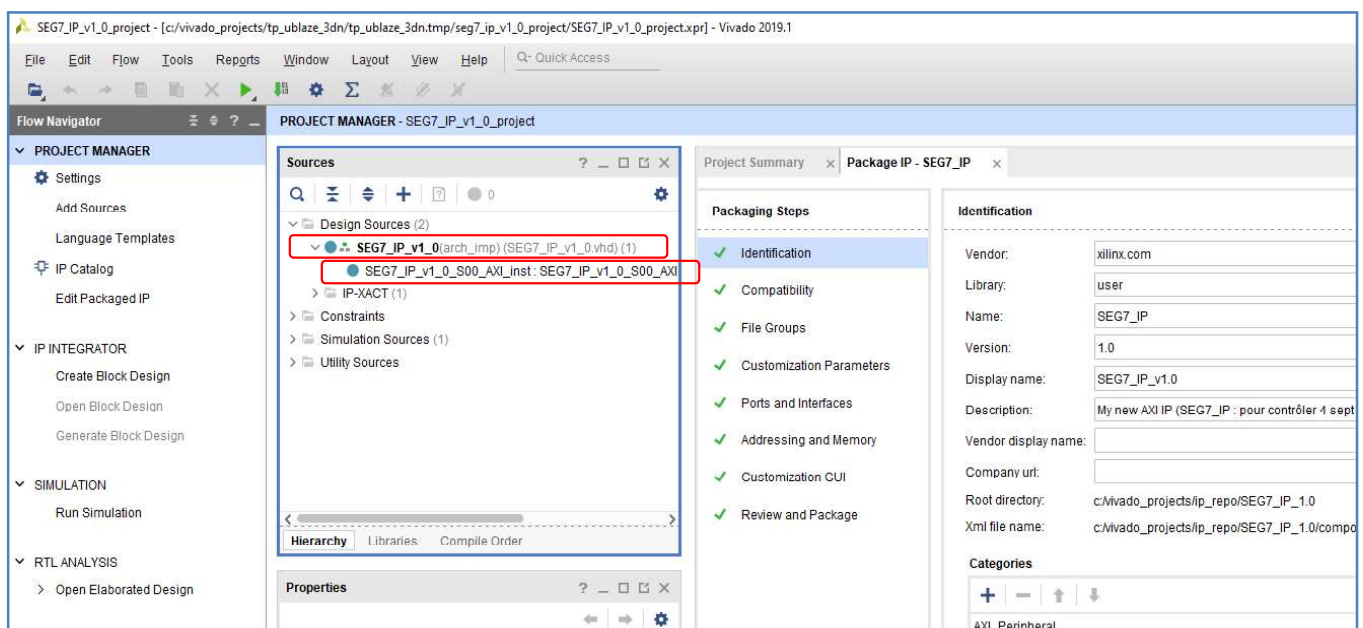
- Dans la fenêtre « Project Manager », sélectionnez « IP Catalog », puis dans la fenêtre principale, recherchez votre nouveau bloc IP dans la section dépôt « User/AXI Peripheral ». Faites un clic droit dessus et sélectionnez « Edit in IP Packager » (voir page suivante)



- Cela ouvrira une nouvelle instance de Vivado avec un projet temporaire juste pour éditer votre bloc IP. Dans la fenêtre suivante, cliquez sur OK :



Dans la fenêtre « Sources » du projet temporaire, il y aura deux fichiers HDL que je vous recommande d'analyser (fichiers disponibles sur moodle.ensea.fr).



A ce stade, voici à quoi ressemble l'IP par défaut (*IP Template*) que vous venez de réaliser. Il s'agit d'une IP intégrant quatre registres, *slv_reg(0 à 3)*, et possédant toutes les interfaces nécessaires pour communiquer avec le bus AXI. Ces interfaces sont décrites en VHDL sous forme de processus (voir le fichier « *SEG7_IP_v1_00_AXI.vhd* »).

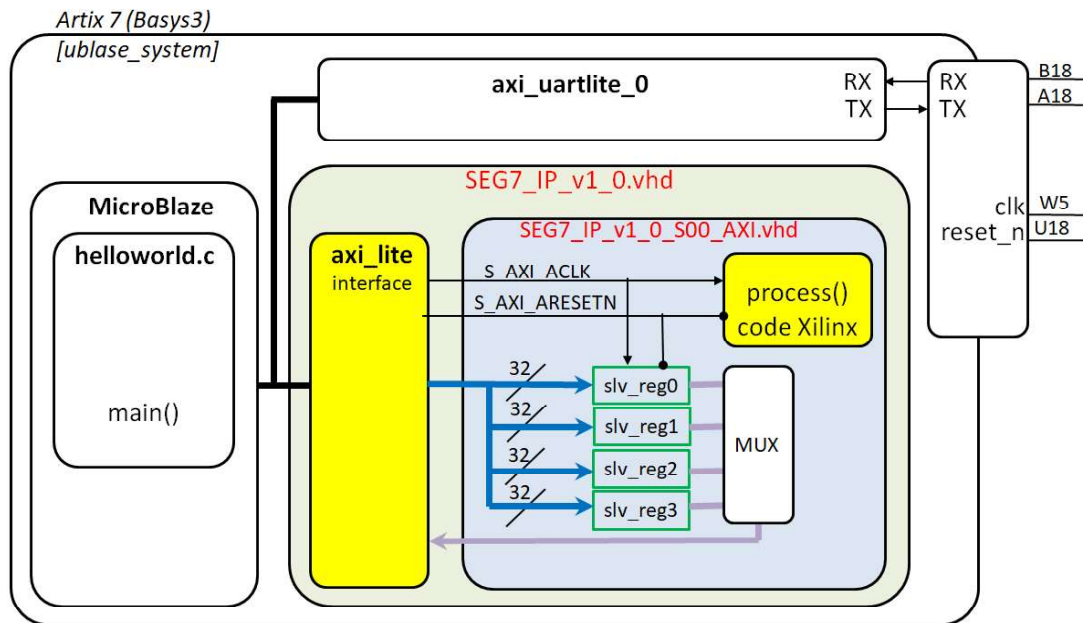
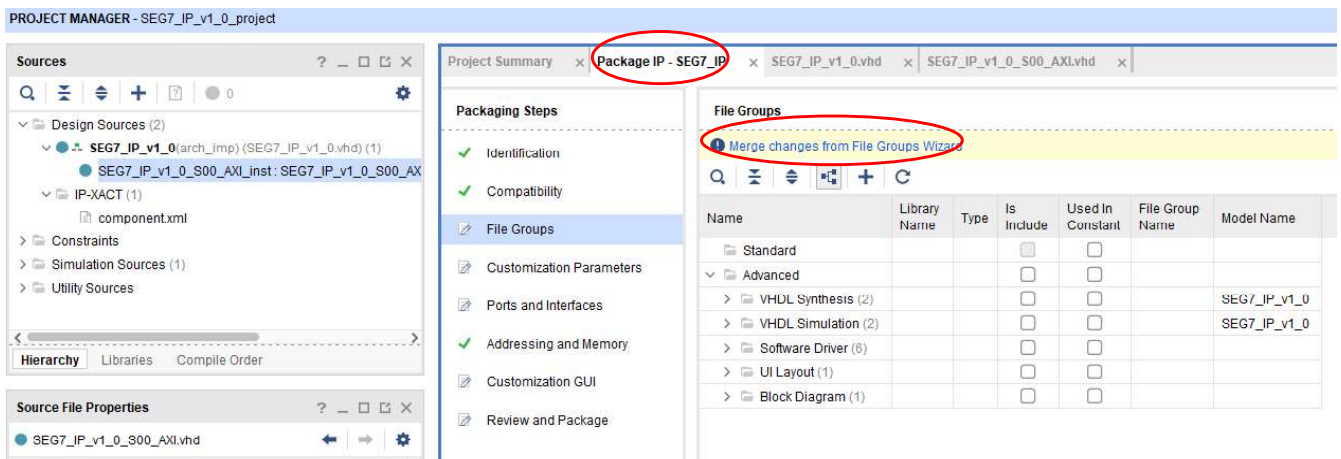


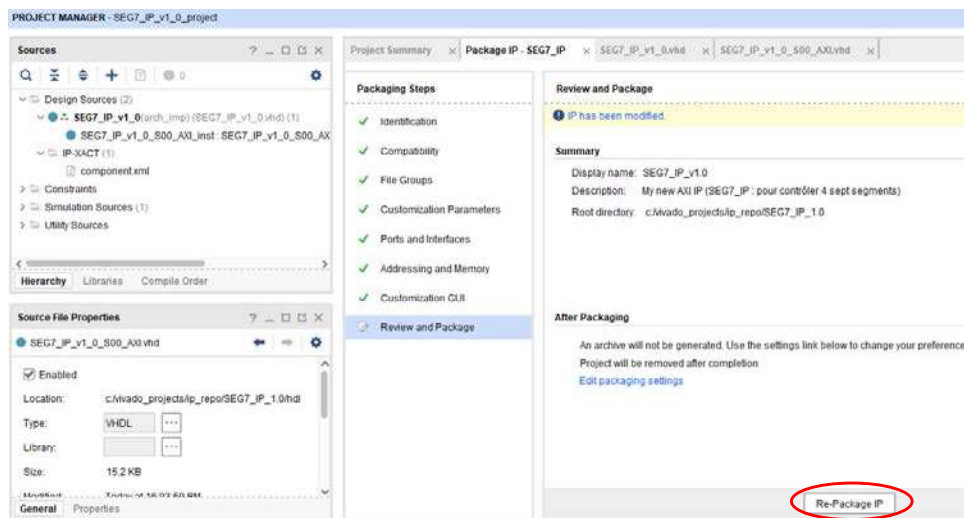
Fig. 1

Editer les 2 fichiers VHDL pour les analyser, repérer les balises délimitant les zones à modifier pour insérer votre propre code (pas pour tout de suite). Nous allons valider l'IP en l'état (sans modification). Les étapes ci-après sont à réaliser à chaque modification apportée au code de l'IP.

- Dans la fenêtre « *Package IP – SEG7_IP* » accepter les mises à jours des étapes non encore validées : « *File Groups, Customization Parameters, ..., Review and Package* »



- Enfin, cliquer sur « *Re-Packager IP* » pour enregistrer les modifications dans le dépôt des IP.

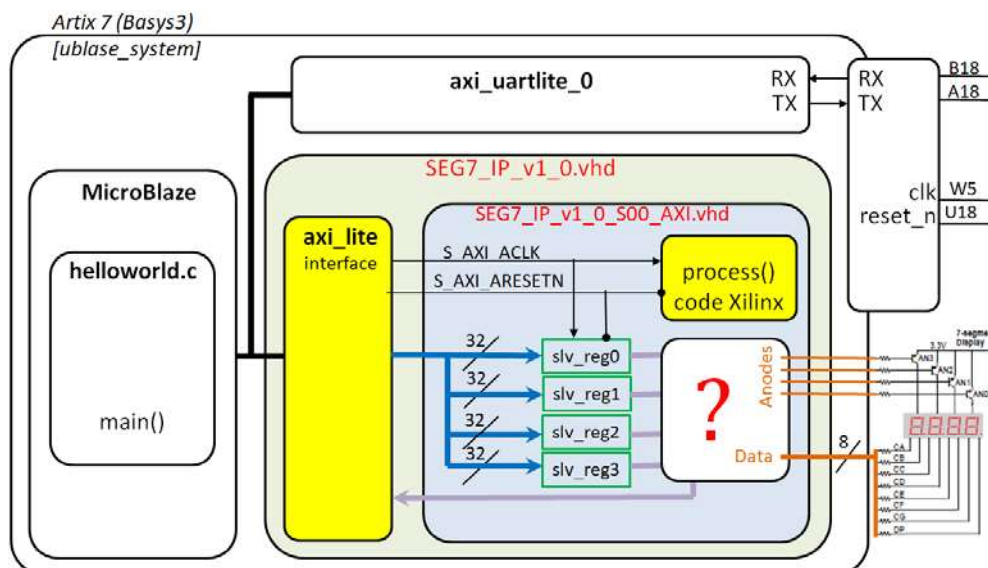


- Dans SDK, écrire une fonction `init_REG()` pour initialiser les 4 registres, `slv_reg(0→3)`, avec les 4 valeurs suivantes : 2, 6, 3, 5. Tester la fonction d'initialisation en affichant le contenu des registres dans la fenêtre SDK Terminal (après une lecture des registres évidemment).

7. Une IP hardware pour contrôler les afficheurs 7 segments

Le squelette de l'IP est maintenant en place et complètement construit. Le travail ultime consiste à compléter le bloc MUX de la figure ci-dessus, **Fig. 1**, afin de contrôler les 4 anodes et les 8 bits de data des afficheurs.

- Ecrire une fonction pour tester le fonctionnement de l'IP en affichant 4321 sur les 4 afficheurs.
- Reprendre le projet software de la partie II pour utiliser le bloc SEG7_IP qui décharge complètement le microcontrôleur **MicroBlaze** du contrôle des 4 afficheurs.



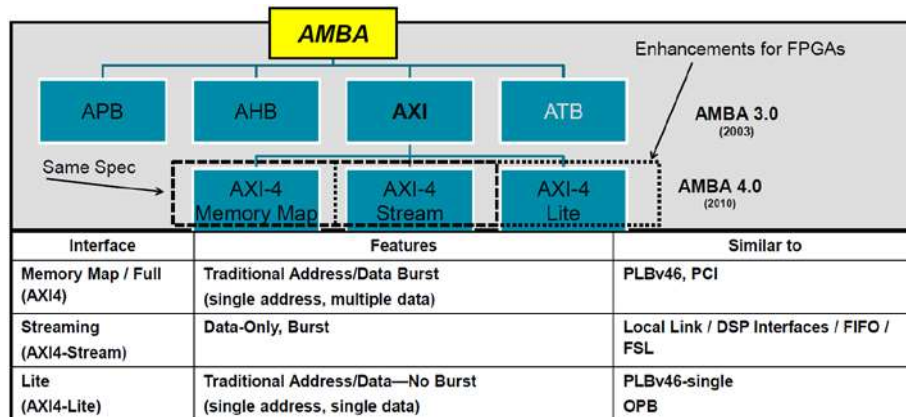
Si le temps vous manque, choisissez un seul des 2 points suivants :

- Rajouter à votre IP un tableau d'entiers, éléments codés sur 8 bits, pour y écrire la table de codage des segments correspondants aux caractères du message. Ecrire une fonction pour tester le fonctionnement. Discuter de la possibilité de libérer complètement **MicroBlaze** de l'animation du message. Confirmez votre solution par une réalisation simple (une cadence fixe de déplacement).
- Remplacer **MicroBlaze** par un processeur **hardcore** pour faire clignoter les LEDs !

ANNEXES

AXI fait partie du bus AMBA (sert pour interface FPGA)

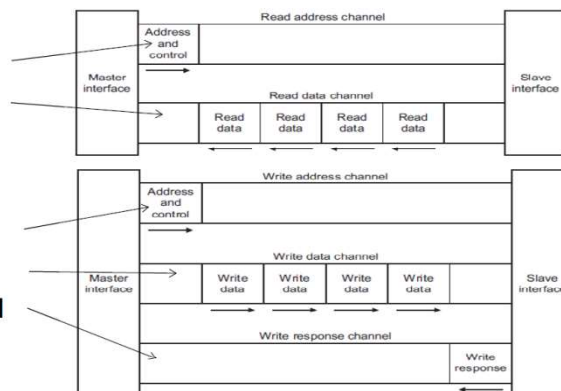
Source : « *Designing a Custom AXI-lite Slave Peripheral* (Rich Griffin, Xilinx Embedded Specialist, Silica EMEA) »



© Copyright Xilinx

Les cinq canaux du protocole AXI-Lite

1. Read Address Channel
2. Read Data Channel
3. Write Address Channel
4. Write Data Channel
5. Write Response Channel



© Copyright Xilinx

AXI4-Lite : Les noms de signaux

```
-- Clock and Reset
S_AXI_ACLK      : in std_logic;
S_AXI_ARESETN  : in std_logic;
-- Write Address Channel
S_AXI_AWADDR    : in std_logic_vector(31 downto 0);
S_AXI_AWVALID   : in std_logic;
S_AXI_AWREADY   : out std_logic;
-- Write Data Channel
S_AXI_WDATA     : in std_logic_vector(31 downto 0);
S_AXI_WSTRB     : in std_logic_vector(3 downto 0);
S_AXI_WVALID    : in std_logic;
S_AXI_WREADY    : out std_logic;
-- Read Address Channel
S_AXI_ARADDR    : in std_logic_vector(31 downto 0);
S_AXI_ARVALID   : in std_logic;
S_AXI_ARREADY   : out std_logic;
-- Read Data Channel
S_AXI_RDATA     : out std_logic_vector(31 downto 0);
S_AXI_RRESP     : out std_logic_vector(1 downto 0);
S_AXI_RVALID    : out std_logic;
S_AXI_RREADY    : in std_logic;
-- Write Response Channel
S_AXI_BRESP     : out std_logic_vector(1 downto 0);
S_AXI_BVALID    : out std_logic;
S_AXI_BREADY    : in std_logic;
```

AXI4-Lite : Transferts en lecture/écriture

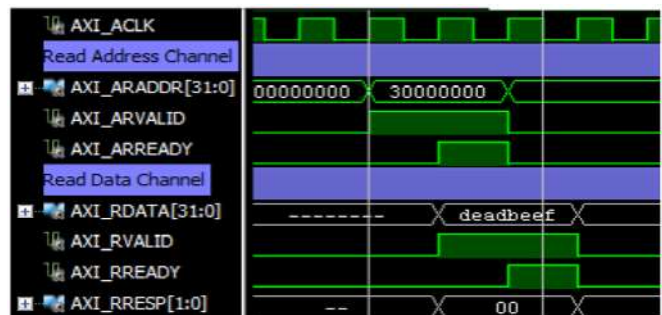
(sources © Silica, Avnet Company)

AXI4-lite Read Address Channel			
Signal Name	Size	Driven by	Description
S_AXI_ARADDR	32 bits	Master	Address bus from AXI interconnect to slave peripheral.
S_AXI_ARVALID	1 bit	Master	Valid signal, asserting that the S_AXI_ARADDR can be sampled by the slave peripheral.
S_AXI_ARREADY	1 bit	Slave	Ready signal, indicating that the slave is ready to accept the value on S_AXI_ARADDR.

Read Transactions

AXI4-lite Read Data Channel			
Signal Name	Size	Driven by	Description
S_AXI_RDATA	32 bits	Slave	Data bus from the slave peripheral to the AXI interconnect.
S_AXI_RVALID	1 bit	Slave	Valid signal, asserting that the S_AXI_RDATA can be sampled by the Master.
S_AXI_RREADY	1 bit	Master	Ready signal, indicating that the Master is ready to accept the value on the other signals.
S_AXI_RRESP	2 bits	Slave	A "Response" status signal showing whether the transaction completed successfully or whether there was an error.

AXI4-lite Response Signalling		
RRESP State [1:0]	Condition	Description
00	OKAY	"OKAY" The data was received successfully, and there were no errors.
01	EXOKAY	"Exclusive Access OK" This state is only used in the full implementation of AXI4, and therefore cannot occur when using AXI4-Lite.
10	SLVERR	"Slave Error" The slave has received the address phase of the transaction correctly, but needs to signal an error condition to the master. This often results in a retry condition occurring.
11	DECERR	"Decode Error" This condition is not normally asserted by a peripheral, but can be asserted by the AXI interconnect logic which sits between the slave and the master. This condition is usually used to indicate that the address provided doesn't exist in the address space of the AXI interconnect.



Write Transactions

AXI4-lite Write Data Channel			
Signal Name	Size	Driven by	Description
S_AXI_WDATA	32 bits	Master	Data bus from the Master / AXI interconnect to the Slave peripheral.
S_AXI_WVALID	1 bit	Master	Valid signal, asserting that the S_AXI_WDATA can be sampled by the Master.
S_AXI_WREADY	1 bit	Slave	Ready signal, indicating that the Master is ready to accept the value on the other signals.
S_AXI_WSTRB	4 bits	Master	A "Strobe" status signal showing which bytes of the data bus are valid and should be read by the Slave.

AXI4-lite Write Response Channel			
Signal Name	Size	Driven by	Description
S_AXI_BREADY	1 bit	Master	Ready signal, indicating that the Master is ready to accept the "BRESP" response signal from the slave.
S_AXI_BRESP	2 bits	Slave	A "Response" status signal showing whether the transaction completed successfully or whether there was an error.
S_AXI_BVALID	1 bit	Slave	Valid signal, asserting that the S_AXI_BRESP can be sampled by the Master.

